

F24-177-D-RastaSaaz

Project Team

Mariam Tahir 21I-0373
Eman Hassan 21I-0862
Asjad Rauf 21I-0493

Session 2021-2025

Supervised by

Mr. Muhammad Aadil Ur Rehman

Co-Supervised by

Mr. Muhammad Farrukh Bashir



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2025

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.3	Modules	2
1.3.1	Module 1	2
1.3.2	Module 2	3
1.3.3	Module 3	3
1.3.4	Module 4	3
1.4	User Classes and Characteristics	4
2	Project Requirements	5
2.1	Use-case/Event Response Table/Storyboarding	5
2.1.1	High-level Use Cases	6
2.1.2	Extended Use Cases	9
2.1.3	Use Case Diagram	18
2.2	Functional Requirements	19
2.2.1	Module 1	19
2.2.2	Module 2	19
2.2.3	Module 4	20
2.2.4	Module 5	21
2.3	Non-Functional Requirements	22
2.3.1	Reliability	22
2.3.2	Usability	22
2.3.3	Performance	22
2.3.4	Security	23
2.4	Domain Model	24
3	System Overview	25
3.1	Architectural Design	26
3.2	Design Models	29
3.3	Data Design	43
4	Conclusions and Future Work	45

References

47

List of Figures

2.1	Use Case Diagram	18
2.2	Domain Model	24
3.1	Box and line Diagram	27
3.2	Architecture Diagram.	29
3.3	Manage Profile Activity Diagram.	31
3.4	Upload Resume Activity Diagram	32
3.5	Analyze Resume Activity Diagram	33
3.6	Explore Jobs Activity Diagram	34
3.7	Refactor Resume Activity Diagram	35
3.8	Rank Resume Activity Diagram	36
3.9	Assess Skill Gap Activity Diagram	37
3.10	Streamline Application Process Activity Diagram	38
3.11	Oversee Application Insights Activity Diagram	39
3.12	Class Diagram	40
3.13	Sequence Diagram	41
3.14	State Transition Diagram	42

List of Tables

2.1.1.Manage profile high-level use case	6
2.1.1.Upload resume high-level use case	6
2.1.1.Analyze resume high-level use case	6
2.1.1.Explore jobs high-level use case	6
2.1.1.Rank resume high-level use case	7
2.1.1.Assess skill gaps high-level use case	7
2.1.1.Streamline application process high-level use case	7
2.1.1.Refactor resume high-level use case	8
2.1.1.Oversee application insights high-level use case	8
2.1.2.Manage profile extended use case	9
2.1.2.Uploading Resume extended use case	10
2.1.2.Analyze Resume extended use case	11
2.1.2.Explore jobs extended use case	12
2.1.2.Rank Resume extended use case	13
2.1.2.Assess skill gap extended use case	14
2.1.2.Refactor resume extended use case	15
2.1.2.Streamline application process extended use case	16
2.1.2.Oversee application insights high-level use case	17

Chapter 1

Introduction

The purpose of this document is to describe RastaSaaz, an automatic resume refactoring system designed to boost job placement by optimizing resumes for specific job requirements, while automating the job application process for maximum visibility and success. There are existing softwares like ResumeGenius, Workable, ResumeGPT which contains features such as resume filtering, parsing, and general formatting capabilities but there is no such software that automatically refactors the resume based on job descriptions. They lack the ability to prioritize the relevant resume sections and remove irrelevant details after comparing the resume with the job descriptions. Our system will fill in this gap by providing a comprehensive solution for resume refactoring and streamline the job application process.

1.1 Problem Statement

In the competitive job market, graduates face many challenges in building a resume that meets Applicant Tracking System (ATS) format. Many job seekers struggle with aligning their resume to specific job descriptions. The manual process of tailoring each resume for different positions is hard and a time consuming task. Existing platforms offer basic features like resume parsing, generation, and skill-based job matching, but lack automated, comprehensive solutions.

1.2 Scope

RastaSaaz is a web based solution designed to assist graduates in crafting optimized, professional resumes tailored to the specific job descriptions. Our system will parse the resume to extract all the relevant information. Once the resume is parsed its skills will be matched with those required in job description, on the basis of skills matched the resumes will be ranked. The resumes with high rank will be automatically refactored, rearranging sections to highlight the most relevant content at the top and removing irrelevant details. Our system narrowly focuses on IT resumes, neglecting to explore different domains with distinct resume styles and technologies. The refactoring process will not include creating projects and experience description by itself, but it would only arrange the project priority wise according to the job description on LinkedIn. Moreover, our system will refine the professional summary of the user to include specific keywords mentioned in the job descriptions. After the refactoring process, our system will apply on multiple jobs using automation.

1.3 Modules

There four modules of our project are listed below:

1.3.1 Module 1

Data Collection and Resume Parsing

In this module, we will be working on documentation, data collection and parsing of the Resume using Python libraries to extract education, experience, skills, projects and categorizing them into different sections.

1. Data Collection. [1], [2].
2. Resume Parsing .
3. Database creation to store resume.

1.3.2 Module 2

Job posts Scraping and Ranking

In this module we will be scraping the job postings from LinkedIn, extracting details from job descriptions, creating the database to store job posting and its parsed fields in it for ranking the resume and assigning a score to the resume.

1. Scraping job postings from Linked-In.
2. Extraction of skills and qualifications from job descriptions.
3. Matching resume with job requirements.
4. Identification of missing skills and projects to assign score.
5. Database creation to store job requirements.

1.3.3 Module 3

Refactoring and Regeneration of Resume

In this module, we will be refactoring the resume by prioritizing the skills, projects, experiences that are most relevant to the job and rearranging sections. We will then be converting this content to a PDF to generate a refactored ATS friendly resume.

1. Prioritizing skills, projects and experience.
2. Rearrangement of sections.
3. Dynamic resume generation in accordance to ATS.

1.3.4 Module 4

Automation of job application

In this module, we will automate the process of applying to jobs on LinkedIn using Selenium. The system will automatically complete job application forms by utilizing stored user data. The final step includes testing the automation functionality to ensure accuracy and reliability before launching the feature on the website.

1. Automation of job applications on Linked-In using Selenium.
2. Automatic completion of application forms using the stored user data.
3. Final testing and website launch.

1.4 User Classes and Characteristics

The user classes and their characteristics for our project are given below:

User class	Description
Job Seekers	Job seekers are individuals who are looking for jobs related to their field of interest and skill set. They are willing to maximize their chances of getting a job. RastaSaaz assists them in refining their resumes, allowing job seekers to automatically apply to multiple positions on LinkedIn, streamlining their job search and increasing their chances of success.

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-case/Event Response Table/Storyboarding

Use Cases:

The use cases of our project are listed below:

1. Manage Profile
2. Upload Resume
3. Analyze Resume
4. Explore Jobs
5. Refactor Resume
6. Rank Resume
7. Assess Skill Gaps
8. Streamline Application Process
9. Oversee Application Insights

2.1.1 High-level Use Cases

Table 2.1.1.1: Manage profile high-level use case

Use case:	Manage Profile
Actor:	Job Seeker
Type:	Primary
Description:	The user can create and manage their personal profile.

Table 2.1.1.2: Upload resume high-level use case

Use case:	Upload Resume
Actor:	Job Seeker
Type:	Primary
Description:	Users can upload their resume in PDF format, which will be parsed by the system, classified into sections like name, email, education, skills, professional summary. This feature enables users to upload their resume which can be further parsed and refactored.

Table 2.1.1.3: Analyze resume high-level use case

Use case:	Analyze Resume
Actor:	Job Seeker
Type:	Primary
Description:	The system will automatically parse the resume by extracting and categorizing key sections of the user's resume, such as summary, experience, projects and skills. This feature will enable users to view their parsed resume.

Table 2.1.1.4: Explore jobs high-level use case

Use case:	Explore Jobs
Actor:	Job Seeker
Type:	Primary
Description:	The system extracts key information from job postings to identify essential skills, qualifications, and requirements. It analyzes the text within job descriptions. This feature enables the user to search for jobs based on position and location.

Table 2.1.1.5: Rank resume high-level use case

Use case:	Rank Resume
Actor:	Job Seeker
Type:	Primary
Description:	User resumes are ranked on the basis of their alignment with specific job descriptions. If a resume lacks essential skills then the user will receive insights on which skills they need to acquire to increase their rank.

Table 2.1.1.6: Assess skill gaps high-level use case

Use case:	Assess Skill Gaps
Actor:	Job Seeker
Type:	Primary
Description:	Identifies gaps between a job seeker's existing skills and the skills required for a specific job. This feature enables job seekers to understand where their qualifications fall short, giving a clear picture of what needs improvement to match the job.

Table 2.1.1.7: Streamline application process high-level use case

Use case:	Streamline Application Process
Actor:	Job Seeker
Type:	Primary
Description:	The parsed resume information will be customized to highlight the most pertinent skills and experiences relevant to the job descriptions. This feature enables the user to initiate the process of refactoring and view the refactored resume.

Table 2.1.1.8: Refactor resume high-level use case

Use case:	Refactor resume
Actor:	Job Seeker
Type:	Primary
Description:	This streamlines the job application process by allowing candidates to submit their refactored resumes to potential employers with minimal manual intervention. The system automatically fills in the necessary application forms, and submits on behalf of the candidate. This feature enhances the efficiency of job search, enabling candidates to focus on interviews rather than spending time on repetitive application tasks.

Table 2.1.1.9: Oversee application insights high-level use case

Use case:	Oversee Application Insights
Actor:	Job Seeker
Type:	Primary
Description:	The user can monitor the status of their application. This feature enables users to stay informed about their job search progress.

2.1.2 Extended Use Cases

Table 2.1.2.1: Manage profile extended use case

Use case section:	UC-1	
Use case name:	Manage Profile	
Primary Actor:	Job seeker	
Description:	The user can create and manage their personal profile which includes changing password or username.	
Pre-condition:	1. The user has registered on the system successfully.	
Post-condition:	1. The user's profile is successfully updated with the modified information.	
Main success scenario:	Actor action	System response
	user selects the "manage profile" option.	
		Change username and change password fields will be displayed along with the current information.
	User changes password or username.	
		System updates the changes made.
	System shows an alert saying "password changed successfully" or "username changes successfully"	
Extensions	1.Profile Data Validation Fails 2.System fails to update profile due to server issue. 3.User cancels profile update.	
Frequency of use	Frequently	

Table 2.1.2.2: Uploading Resume extended use case

Use case section:	UC-2	
Use case name:	Uploading Resume	
Primary Actor:	Job seeker	
Description:	The job seeker uploads a resume in PDF format, which the system will parse automatically.	
Pre-condition:	1.The job seeker is logged into the system. 2.The job seeker has a resume in PDF format.	
Post-condition:	1.The resume is successfully updated. 2.The parsed resume information is saved in the system's database.	
Main success scenario:	Actor action	System response
	Navigates to the "resume upload" section.	
		displays the "upload" option.
	clicks "upload" button	
	uploads a resume in PDF format.	
		validates the format of the uploaded file.
		Saves resume in database.
		displays a success message that the resume has been uploaded.
Extensions	1. The job seeker uploads a resume in a format other than PDF. 2. The user uploads some other document by mistake.	
Frequency of use	Occasionally	

Table 2.1.2.3: Analyze Resume extended use case

Use case section:	UC-3	
Use case name:	Analyze Resume	
Primary Actor:	Job seeker	
Description:	The system will automatically parse the resume by extracting and categorizing key sections of the user's resume. Users will then be able to view this parsed resume.	
Pre-condition:	1. The user has already uploaded their resume in the system.	
Post-condition:	1. Parsed resume is displayed with key sections such as experience, education, skills etc.	
Main success scenario:	Actor action	System response
	Click on the "Analyze Resume" button.	
		Retrieves resume from database.
		Extracts relevant sections from the resume.
		Categorize different sections.
		Parsed resume data stored in database.
		Retrieves the parsed resume data from the database.
		Displays the parsed data in a structured format.
	scroll through different sections of parsed resume	
Extensions	1. Unable to parse resumes due to challenges of different resume structure.	
Frequency of use	Occasionally.	

Table 2.1.2.4: Explore jobs extended use case

Use case section:	UC-4	
Use case name:	Explore jobs	
Primary Actor:	Job seeker	
Description:	The job seeker searches for jobs by applying various filters such as position and location. The system will then retrieve a list of jobs according to that.	
Pre-condition:	1.The job seeker is logged into the system. 2.The job seeker is allowed personalized search results.	
Post-condition:	1. The system presents a list of relevant jobs.	
Main success scenario:	Actor action	System response
	Navigates to the “Search Jobs” section.	
		Displays the search interface input fields.
	Enters each criteria into the input fields	
	Click on the “Search” button to initiate the search.	
		Validates the input search criteria for completeness.
		retrieve job listings from the database.
		The system presents the search results in a structured format.
Extensions	1. The job seeker enters specific search criteria but no jobs match that criteria.	
Frequency of use	Frequently	

Table 2.1.2.5: Rank Resume extended use case

Use case section:	UC-5	
Use case name:	Rank Resume	
Primary Actor:	Job seeker	
Description:	User resumes are ranked on the basis of their alignment with specific job descriptions. If a resume lacks essential skills then the user will receive insights on which skills they need to acquire to increase their rank. Once the resumes are ranked the user will be able to view them.	
Pre-condition:	1. The user has uploaded a resume to the system . 2. System has parsed the resume	
Post-condition:	The user can view the ranking score of their resume.	
Main success scenario:	Actor action	System response
	Selects the “check resume rank” button.	
		Verify if the job seeker has an uploaded resume that has been analyzed.
		analyzed resume will be proceeded for ranking.
		Calculate rank on given criteria
		Store the rank in database.
		Retrieves the rank of the resume from the database.
		Displays the result of that ranked resume
Extensions	1. System Error while retrieving ranking data 2. No parsed resume found. 3. No job postings found.	
Frequency of use	Frequently	

Table 2.1.2.6: Assess skill gap extended use case

Use case section:	UC-6	
Use case name:	Assess skill gap	
Primary Actor:	Job seeker	
Description:	The job seekers can view their skill gaps which the system will identify by matching their resume with the specific job description.	
Pre-condition:	1.The job seeker is logged into the system. 2.The job seeker has uploaded their resume. 3.The job seeker has selected a job role.	
Post-condition:	1. The system displays the identified missing skills relative to the selected job role.	
Main success scenario:	Actor action	System response
	Navigates to “View skill gaps”..	
	retrieves the relevant resume data from the database	
		Retrieves the relevant job description data from the database
		Compares the skills listed in the resume with the required skills in the job description
		Compiles a list of skill gaps in a structured way
		Display the listings of gaps that need to be filled.
Extensions	1. Failed to retrieve data from database.	
Frequency of use	Occasionally	

Table 2.1.2.7: Refactor resume extended use case

Use case section:	UC-8	
Use case name:	Refactor resume	
Primary Actor:	Job seeker	
Description:	The parsed resume information will be customized to highlight the most pertinent skills and experiences relevant to the job descriptions. This feature will give users a newly generated resume with their refactored content in it.	
Pre-condition:	1.The user has uploaded a resume to the system. 2.The system has successfully parsed the resume. 3.The system has access to job descriptions from linkedIn.	
Post-condition:	1.A refactored version of the resume is created based on job description analysis. 2.The refactored resume is stored in the system's database.	
Main success scenario:	Actor action	System response
	Click on the "Generate Refactored Resume" button.	
		Verifies the resume is uploaded.
		Retrieves the uploaded resume from the database.
		Fetches job description from LinkedIn.
		Analyzes the resume with skills listed in the job description.
		Generates the refactored resume by highlighting most relevant content.
		Stores the refactored resume in the database.
		Fetches the resume from the database.
		displays resume on the user interface.
	download the resume	
Extensions	1.System displays a message that no original resume is available for refactoring. 2.Failed to parse resume or job description. 3.System error while generating a refactored resume.	
Frequency of use	Frequently	

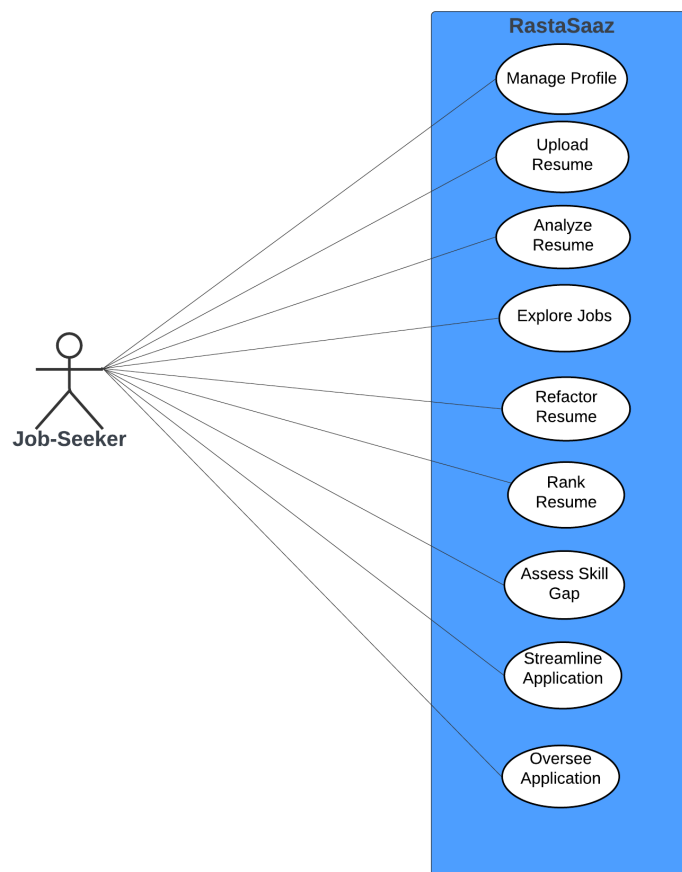
Table 2.1.2.8: Streamline application process extended use case

Use case section:	UC-9	
Use case name:	Streamline application process	
Primary Actor:	Job seeker	
Description:	The job seeker can apply to multiple jobs automatically. The system will submit applications through LinkedIn for streamlining the process.	
Pre-condition:	1.The job seeker is logged into the system. 2.The job seeker has uploaded the resume and identifies job postings. 3.The job seeker has their LinkedIn account.	
Post-condition:	1. The system successfully applies to the selected job on behalf of the user. 2.The job seeker receives confirmation of submitted applications.	
Main success scenario:	Actor action	System response
	Navigates to “apply jobs”	
		Retrieves and displays a list of job postings
	Views the list of job postings.	
	Selects multiple jobs to apply to.	
		Matches resumes with job descriptions.
	Redirect to LinkedIn’s login page not already logged in.	
		Automatically fill in the application fields.
		submits applications
		processes the submission
	View the confirmation message.	
Extensions	1.The job seeker finds no suitable jobs. 2.The job seeker encounters errors which indicates that the application cannot be processed.	
Frequency of use	Frequently	

Table 2.1.2.9: Oversee application insights high-level use case

Use case section:	UC-10		
Use case name:	Oversee Application Insights		
Primary Actor:	Job Seeker		
Description:	The user can monitor the status of their application.		
Pre-condition:	1.The user has applied for job through the system.		
Post-condition:	1.System displays detailed information about each application, including the job title, company name the user has applied to.		
Main success scenario:	Actor action	System response	
	Selects the “Monitor job applications” option from their dashboard		
		Retrieves the job application from the database.	
		Displays a list of job applications	
	View the applications submitted		
Extensions	1. No Job application found 2.System error while retrieving application data. 3.User has not logged into their linkedIn account.		
Frequency of use	frequently		

2.1.3 Use Case Diagram



2.2 Functional Requirements

Following are the functional requirements of our proposed project

2.2.1 Module 1

Data Collection and Resume Parsing

- **Feature: Data Collection**

1. FR1: The system shall allow users to upload resumes in PDF format.

- **Feature: Resume Parsing**

1. FR2: The system shall convert the resume from PDF format into JSON format.
2. FR3: The system shall extract key details from the resume.
3. FR4: The system should categorize details into sections in JSON format.

- **Feature: Database creation to store resume**

1. FR5: The system shall store extracted details in a structured database.
2. FR6: The system should assign a unique ID to each resume.
3. FR7: The system must allow for easy retrieval of resume data.

2.2.2 Module 2

Job posts Scraping and Ranking Module

- **Feature: Scraping job postings**

1. FR1: The system shall scrape IT-related jobs based on predefined filters.

- **Feature: Extraction of skills and qualifications from job descriptions**

1. FR2: The system shall extract required skills, qualifications from the job descriptions.

- **Feature: Matching resume with job requirements and assigning score**

1. FR3: The system shall compare resume details with job descriptions.
2. FR4: The system must assign rank to each resume based on how well it matches the job requirements.

- **Feature: Identification of missing skills and projects in the resume**

1. FR1: The system shall analyze the candidate's resume and match with job description to identify missing skills.
2. FR2: The system should highlight skill gaps to the user.

- **Feature: Database creation to store job requirements**

1. FR5: The system shall store job requirements in a structured database

2.2.3 Module 4

Refactoring of Resume

- **Feature: Prioritizing skills, projects and experience**

1. FR1: The system shall prioritize relevant skills, projects, and experience from resumes by analyzing job descriptions.

- **Feature: Rearrangement of sections**

1. FR2: The system shall automatically rearrange the resume sections based on ATS.
2. FR3: The system shall allow users to preview the rearranged resume.

- **Feature: Dynamic resume generation in accordance to ATS**

1. FR4: The system must generate ATS compliant resumes ensuring that it includes keywords
2. FR5: The system shall generate resumes tailored to different job descriptions.
3. FR6: The system should allow users to download the resume into PDF

2.2.4 Module 5

Automation of job application

- **Feature: Automation of job applications on Linked-In using Selenium**

1. FR1: The system shall automate the job application process for relevant job postings.

- **Feature: Automatic completion of application forms using the stored user data**

1. FR2: The system shall extract data from resume to auto-fill job applications
2. FR3: The system shall use the refactored resume to apply for jobs

- **Feature: Final testing and website launch**

1. FR4: The system shall be tested for automation of job applications

2. FR5: The system shall be fully functional and accessible to users by the website launch.

2.3 Non-Functional Requirements

Given below are the non-functional requirement of our project:

2.3.1 Reliability

The RastaSaaz system shall have a mean time between failures of at least 400 hours. Failures are defined as any event where the system becomes unresponsive, fails to parse resume, inaccurately matches job description or fail to complete job applications. If the system fails, the system shall maintain user data information, ensuring no data loss to ongoing processes, such as resume parsing, refactoring or job application. The system shall resume seamless operations upon recovery. Our system will implement a comprehensive error detection strategy including anomaly detection algorithms for addressing critical issues such as failed resume parsing or job application process.

2.3.2 Usability

The RastaSaaz shall provide an intuitive dashboard where users can upload resume, view skill gap analysis, course recommendations and download refactored resumes. The error messages displayed shall be clear and concise along with suggestions to correct the issue such as uploading resume in PDF format. The system shall meet the accessibility standards to ensure that users with disabilities can navigate easily.

2.3.3 Performance

The system shall parse and analyze a resume within 15 seconds of upload, with 50% of resumes being processed within this time frame. The job de-

scription parsing and resume matching algorithms shall complete within 10 seconds per job description, ensuring fast and efficient analysis for users.

2.3.4 Security

All sensitive user data, including personal information, resumes, and job applications should be protected against unauthorized access. Moreover, field level encryption in MongoDB will be used to ensure sensitive data, such as personal detail is automatically encrypted before storage and decrypted when accessed with the appropriate permission, enhancing data security.

2.4 Domain Model

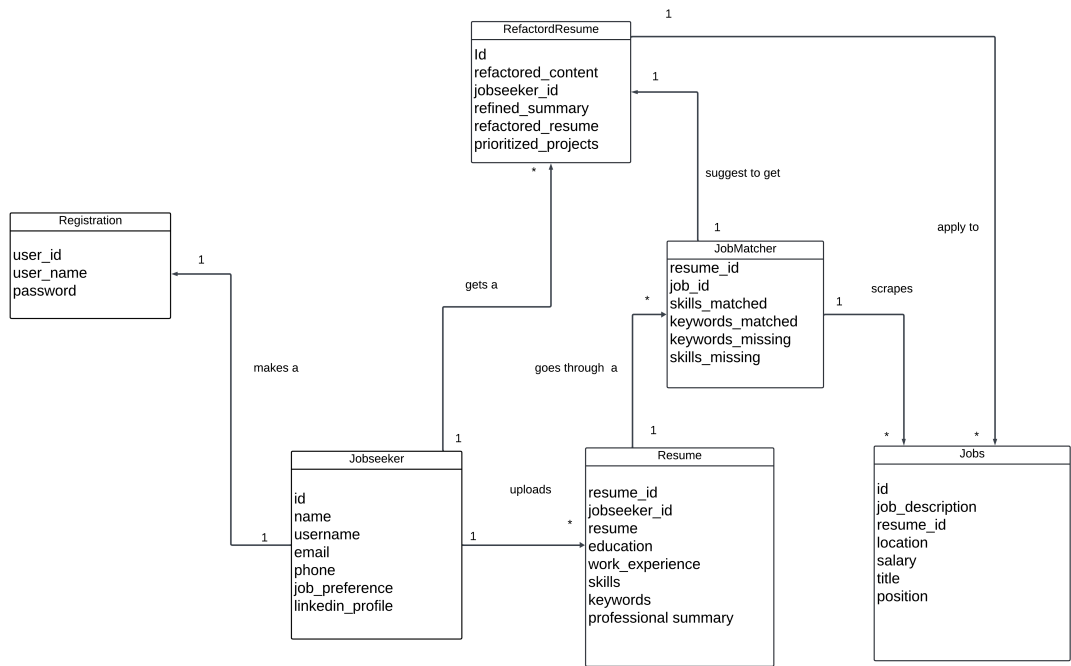


Figure 2.2: Domain Model

Chapter 3

System Overview

Rasta Saaz is an AI-driven Resume refactoring system for fresh graduates and job seekers. Its primary objective is to automate the task of applying to multiple IT related jobs by refactoring the resumes according to the job descriptions on linkedin, with minimum human intervention.

Background

RastaSaaz emerges in response to the challenge of manually tailoring resumes for each job application, a process that is both time-consuming and labor-intensive. Despite the increasing demand for personalized resumes, there is a notable lack of systems that automate this process. To address this gap, our platform automates resume refactoring, enabling users to efficiently adapt their resumes to meet specific job requirements without manual intervention.

Context

The System is developed to cater the growing demand of the automatic resume refactoring system. Our system will generate a personalized and specific refactored resume which would enhance the chances of getting a job.

Design

RastaSaaz design divides the application into distinct layers which are presentation, business logic, data layer. Each of these layers will focus on one

task making RastaSaaz easy to develop, maintain and extend. This design promotes scalability and re-usability and ensures seamless user experience.

Functionality

RastaSaaz user-friendly interface will enable the fresh graduates and other job seekers to manage their profile, upload a Resume PDF which will be analyzed and matched with preferred job positions on linkedIn. Moreover, Our system will automatically restructure and generate a new ATS-friendly refactored resume which will then be automatically used to apply to jobs. If the resume ranking algorithm generates a low score, the system will conduct a skill gap analysis and if the rank is high only then the refactoring will be done.

3.1 Architectural Design

For RastaSaaz, we have chosen a modular program structure, where we will be implementing each module separately. These modules are divided such that they represent the main functionality of our system. The modules are Data collection and Resume Parsing, Job post scraping and Ranking, Refactoring and Regeneration, Automation of job applications. The relationship between the modules and how they work together is explained below.

1. Data collection and Resume Parsing Module:

- This module collects and analyzes the resume data and store it in a structured format for later use
- It interacts with the Job Post scraping module for similarity analysis and to assign a score to each parsed resume.

2. Job Post Scraping and Ranking Module:

- This module scrapes the IT related job posting from linkedIn using linkedIn API, matches them with the job seekers resume and assign them a rank.

- This module interacts with the refactoring and regeneration module to refactor the high ranking resume and generate a tailored version before applying to a job.

3. Refactoring of Resumes Module

- This module provides users a platform to automatically restructure their resumes, to create an ATS-friendly resume to align with industry trends.
- This module interacts with the automation of the job application module by providing the refactored resume so that the automatic job application process can proceed.

4. Automation of job application Module

- This module automatically completes the job application using the refactored resume and then final testing is performed.

Box and Line Diagram



Figure 3.1: Box and line Diagram

Multi Layer Architecture

A multi-layer architecture is ideal for RastaSaaz due to its structured separation of concerns. In this architecture, the presentation layer provides an intuitive user interface for job seekers. It provides an intuitive user interface for job seekers, facilitating seamless interactions with system functionalities. A smooth user experience is delivered in the presentation layer, which manages the user interface and interaction elements.

The Application layer encompasses essential modules such as job-seeker's profile management, resume parsing, job scraping, skill gap analysis, job matching, resume ranking and resume refactoring. These modules will contain the business logic required to make the function on the interface operate smoothly.

The data layer efficiently handles database operations, storing user profiles, resumes, job descriptions, and matching data, ensuring data integrity and optimized access.

Additionally, while the API access component interfaces with external services to enrich functionality—such as fetching job listings—it operates outside the core layers, allowing seamless integration without complicating the architecture. Moreover, an audit log is crucial for tracking user actions and changes, enhancing accountability and troubleshooting, while security features protect sensitive user data and ensure compliance with best practices, making the overall system robust and reliable.

Multi-layer Architecture Diagram

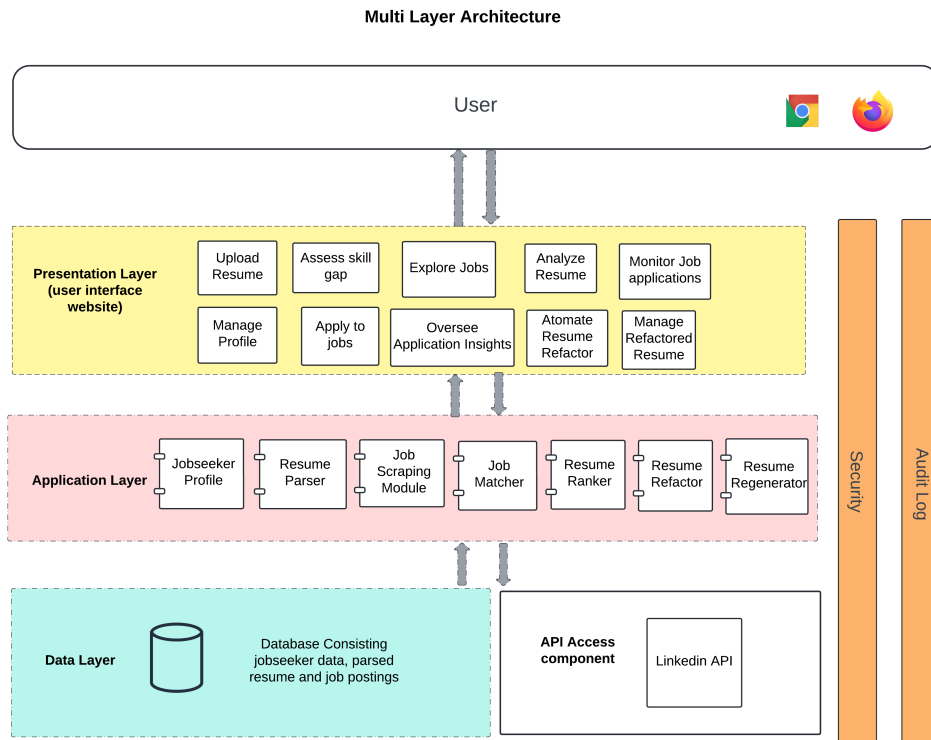


Figure 3.2: Architecture Diagram.

3.2 Design Models

The development of RastaSaaz is better suited to an Object-Oriented Programming (OOP) approach rather than a procedural one. The system is composed of modules that require interaction and OOP structures the system around objects, making it more efficient to develop, maintain, and scale. Each module in RastaSaaz can be modeled as an object, encapsulating its relevant data (attributes) and functionality (methods). This modular approach enhances manageability by promoting re-usability, flexibility, and clear interactions between the various models and modules, allowing the platform to evolve and grow seamlessly over time.

Design Models for Object Oriented Development Approach

The applicable models for the project using object oriented development approach may include:

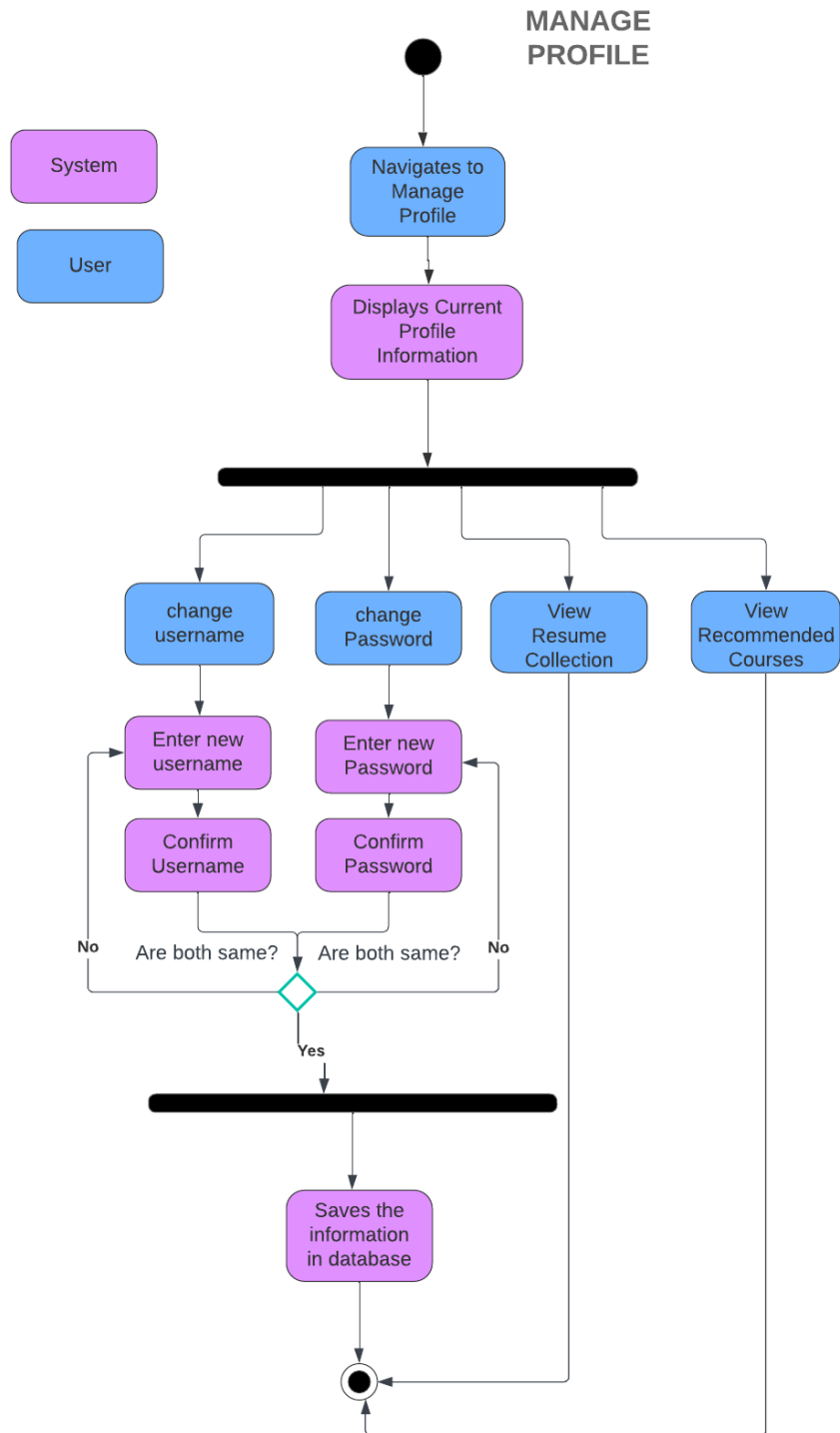


Figure 3.3: Manage Profile Activity Diagram.

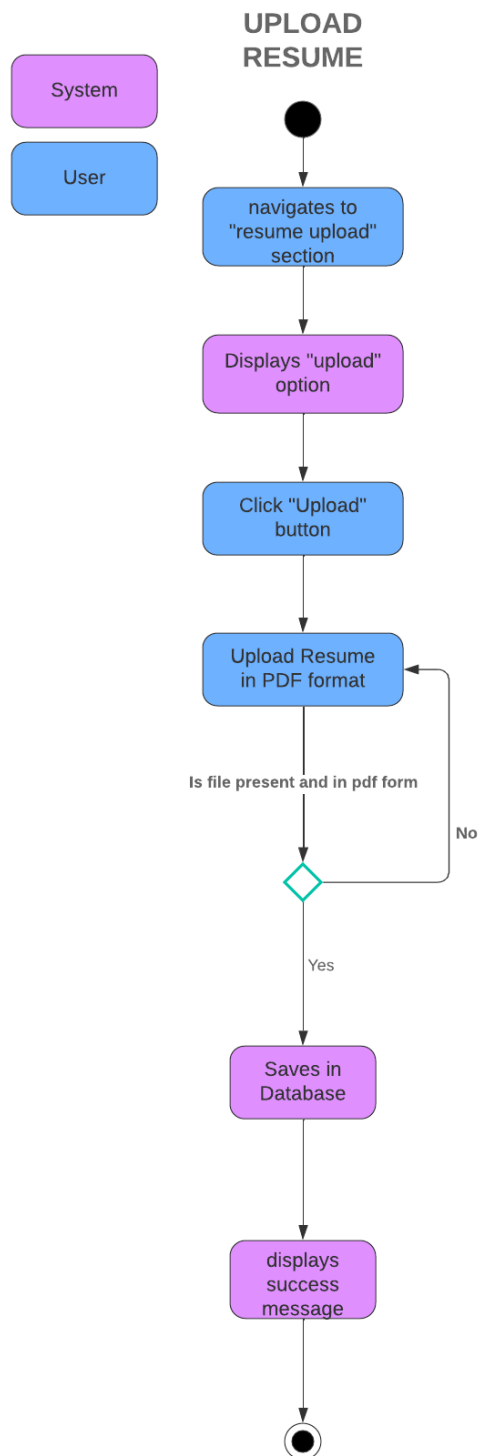


Figure 3.4: Upload Resume Activity Diagram

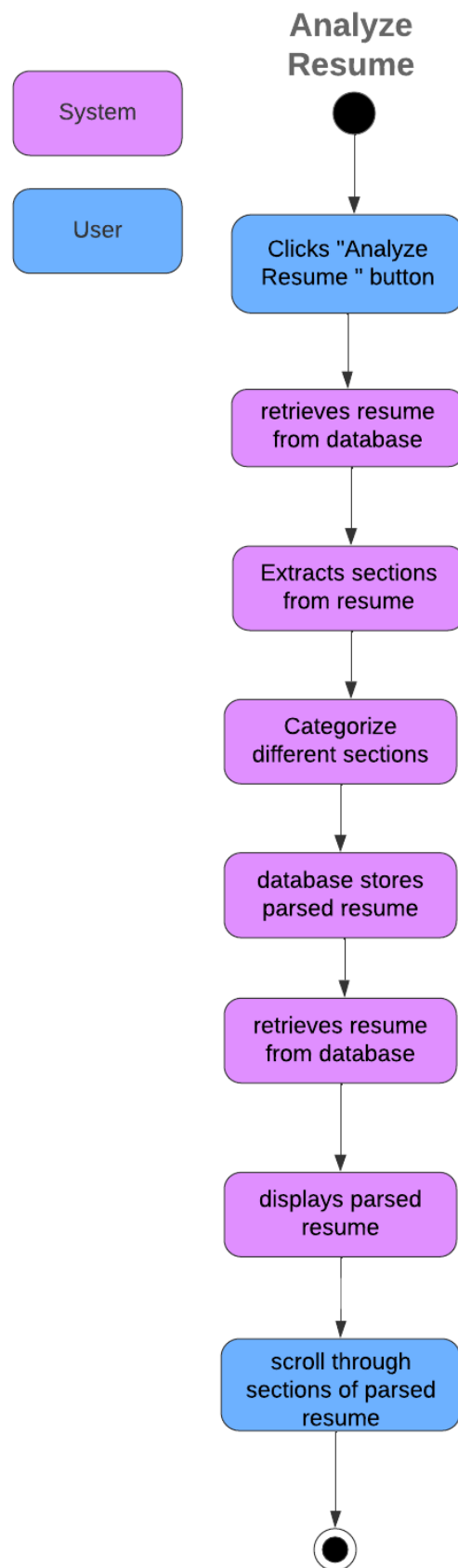


Figure 3.5: Analyze Resume Activity Diagram

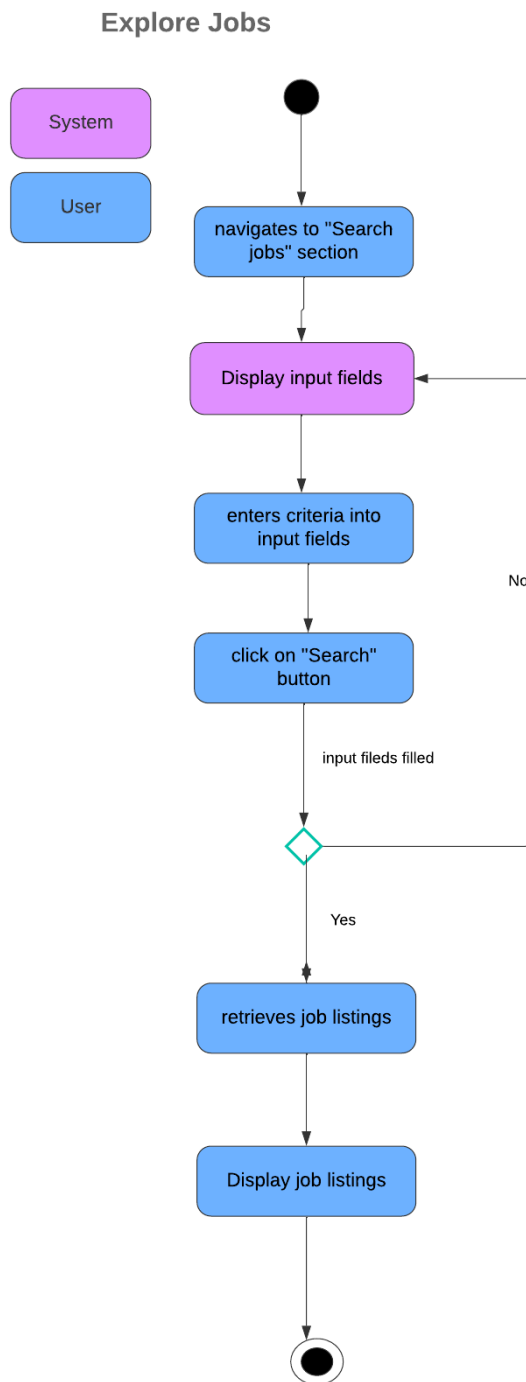


Figure 3.6: Explore Jobs Activity Diagram

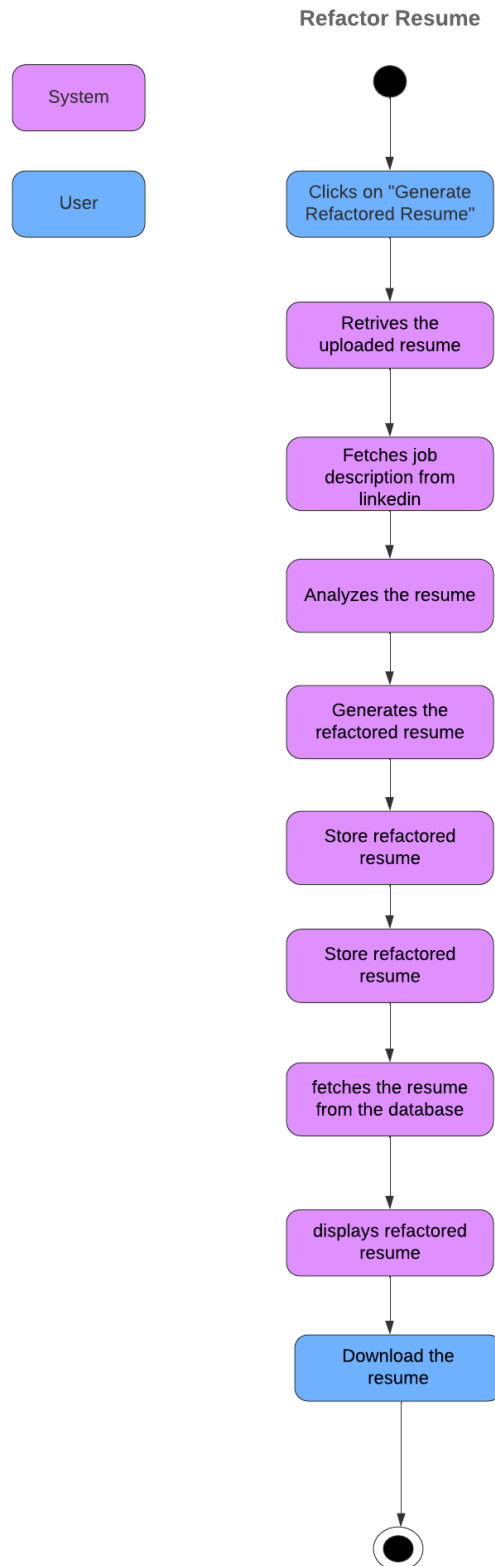


Figure 3.7: Refactor Resume Activity Diagram

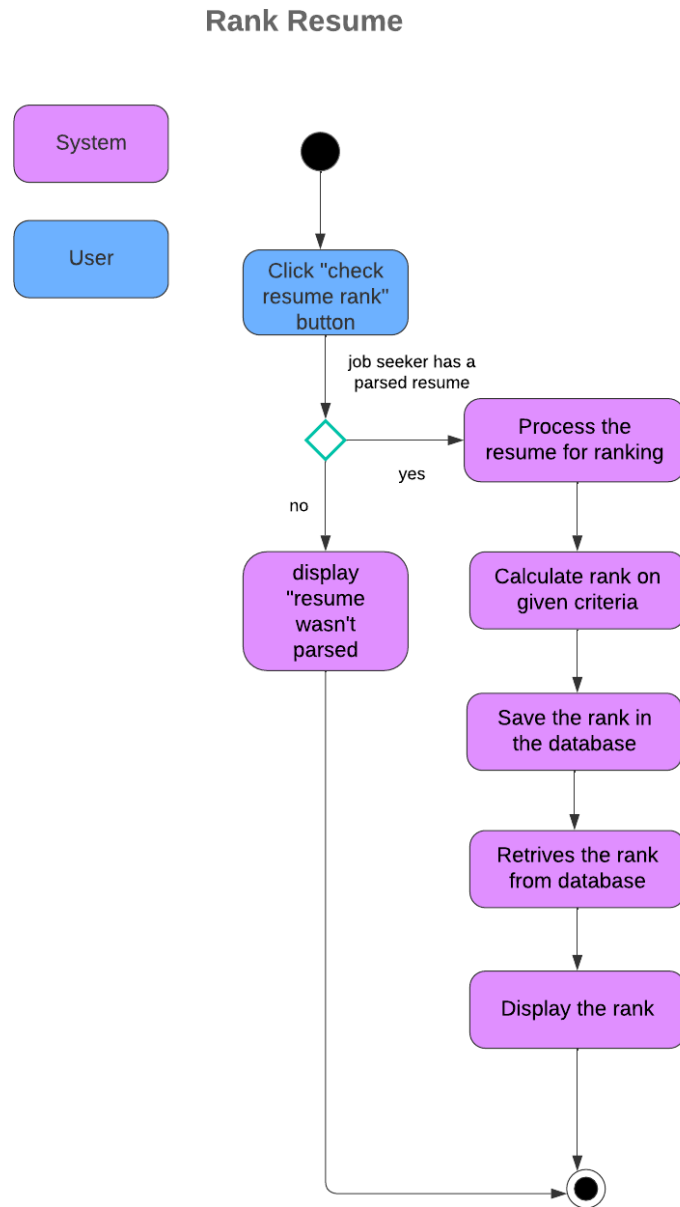


Figure 3.8: Rank Resume Activity Diagram

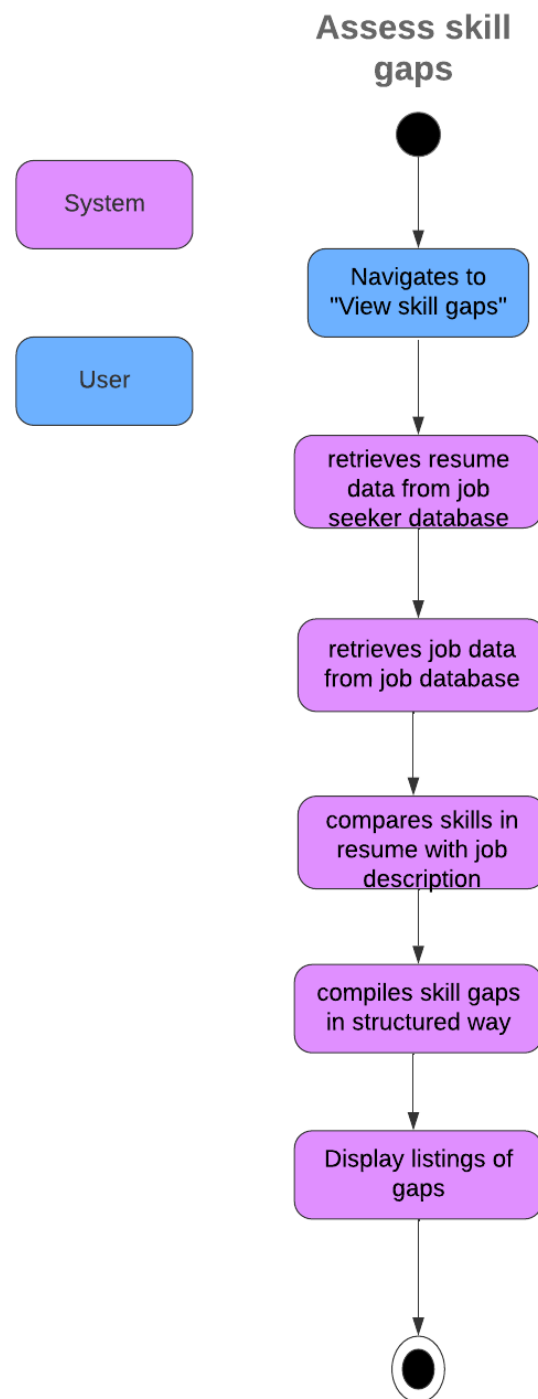


Figure 3.9: Assess Skill Gap Activity Diagram

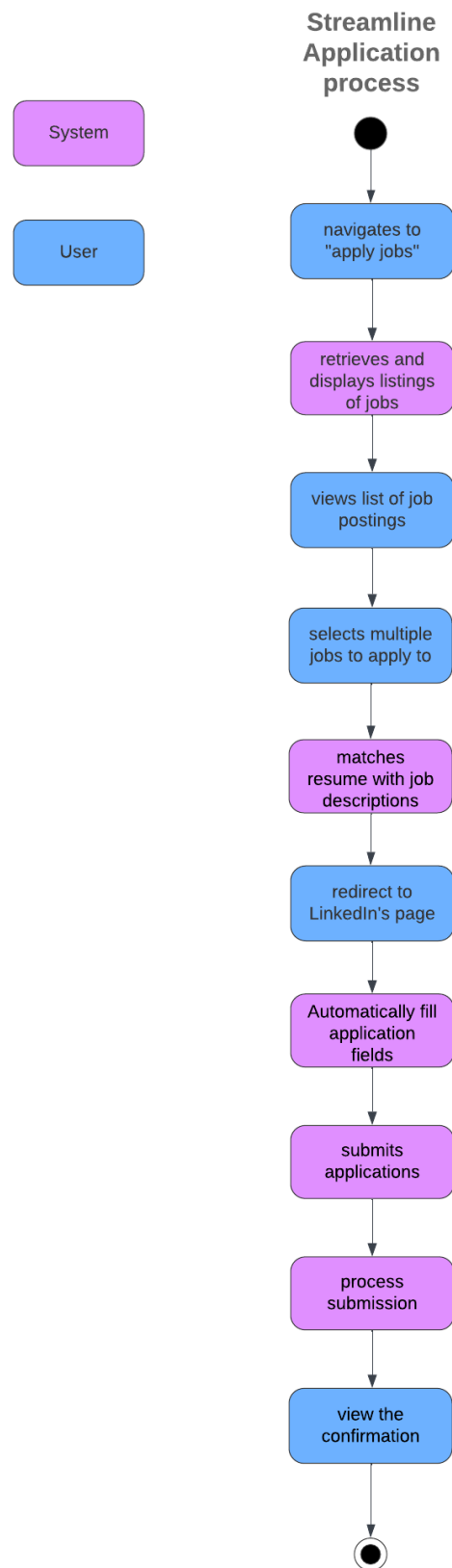


Figure 3.10: Streamline Application Process Activity Diagram

Oversee Application Insights

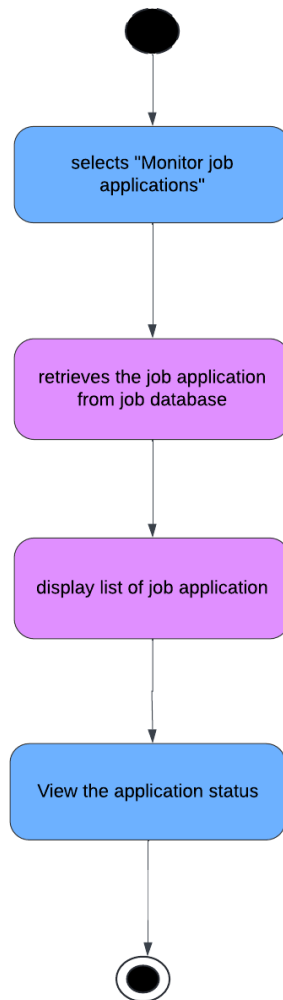


Figure 3.11: Oversee Application Insights Activity Diagram

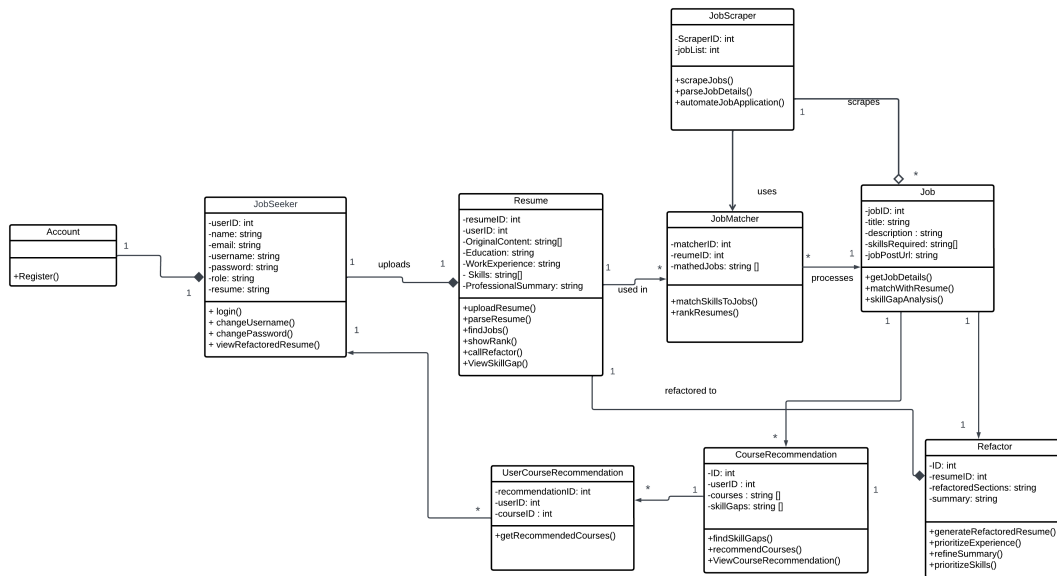


Figure 3.12: Class Diagram

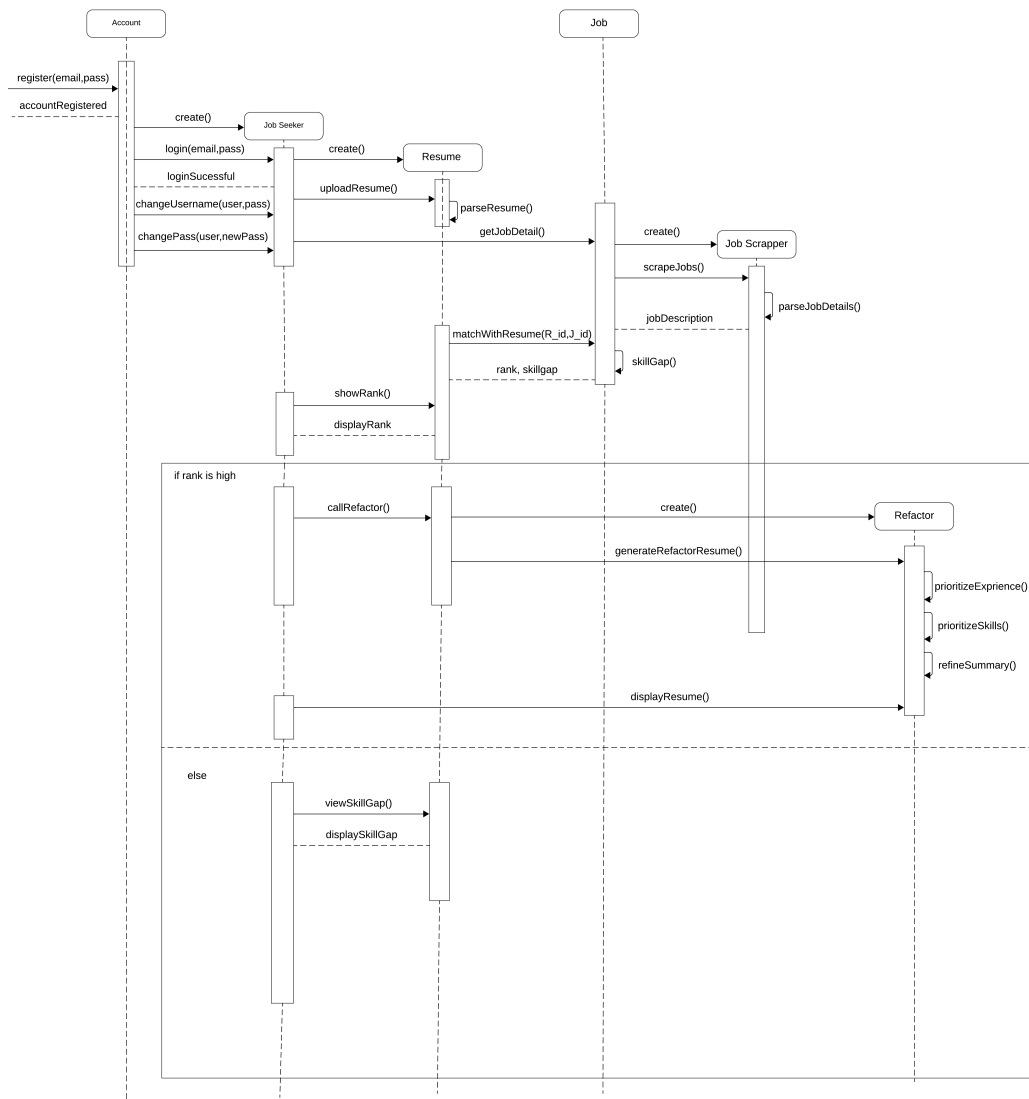


Figure 3.13: Sequence Diagram

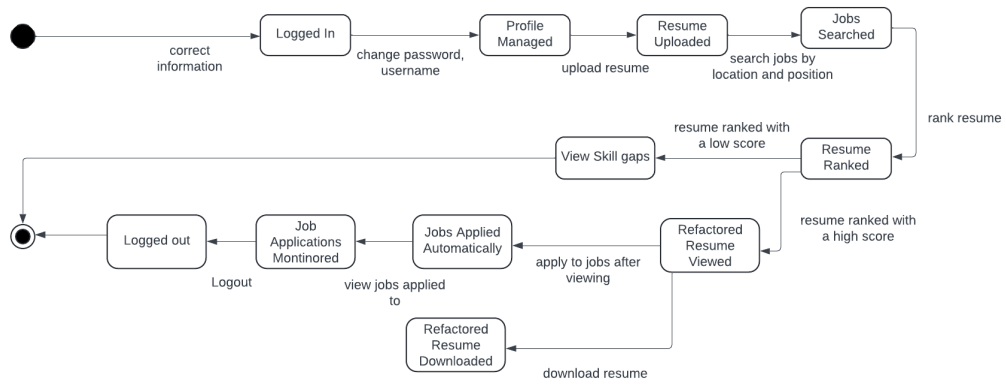


Figure 3.14: State Transition Diagram

3.3 Data Design

The information domain in RastaSaaz contains Job seeker profile, Refactored Resume and jobs description which is transformed into data structures to be stored effectively. Following points show how this data is stored, processed, and organized.

1. Job seeker Profile

- Stored in the database, which contains username, password, email, PDF resume.
- Processed for changing username, password and analyzing the resume.

2. Refactored Resume

- Generated through the refactor resume module.
- Stored in the database, containing section wise attributes in the table.
- They are processed to generate new resume.

3. Job Descriptions

- Scraped from linkedIn
- This data which includes the job title, description, requirements and qualification is stored in the database.
- This will be processed to clean it by removing stop words, lemmatization and other NLP techniques to implement resume. matching.

Databases or Data Storage Items

1. **Unified RastaSaaz Database** Stores all user-related information and initially uploaded resumes along with the refactored resumes sections for each individual. It will also store the parsed job description from linkedIn, organized into sections for skill matching and gap analysis. Also, the rank assigned to each resume will also be stored in the database. This unified structure ensures efficient data retrieval, storage, and updates for all key components of system.

Chapter 4

Conclusions and Future Work

In conclusion, RastaSaaz marks a significant milestone in resume refactoring, providing graduates with a streamlined platform to enhance their job application process. By integrating advanced natural language processing (NLP) techniques and machine learning models, RastaSaaz empowers users to refine their resumes, identify skill gaps and apply to jobs.

1. **Enhanced Skill Gap Analysis:** Future iterations of RastaSaaz could incorporate more sophisticated algorithms to provide deeper insights into missing skills, offering personalized learning pathways and specific job role recommendations based on users' current expertise.
2. **Expanding Industry Coverage:** While RastaSaaz is currently focused on IT careers, future developments could expand the platform to include additional industries, such as healthcare, finance, and engineering, making the system more versatile and accessible to a broader range of users.
3. **Continuous User Feedback and Iterative Improvement:** Regular user feedback and data-driven improvements will be key to refining the RastaSaaz platform. Ongoing updates based on evolving job market trends and user insights will ensure RastaSaaz remains competitive, delivering relevant and valuable services to its users.

Bibliography

- [1] <https://www.kaggle.com/datasets/snehaanbhawal/resume-dataset?select=>
- [2] <https://www.kaggle.com/datasets/anuvagoyal/resume-pdf/data?select=A>